

ASTALAKSHIMI MATRIMONY

HIGH-LEVEL DESIGN (HLD), SYSTEM ARCHITECTURE & CODEBASE AUDIT

A Complete Technical Blueprint, Data Flow Tracing, and Repository Health Audit

Target Stack:	Next.js 16 (App Router) + React 19 + NestJS + Drizzle ORM + PostgreSQL + AWS S3
Repository Architecture:	pnpm Workspaces Monorepo + Turborepo Build Pipeline
Database Schema:	18 Relational Tables (PostgreSQL / AWS RDS)
Document Version:	2.0.0 (Comprehensive Repository Audit Edition)
Audit Date:	August 2026
Author / Role:	Senior Principal Software Architect & Lead Security Auditor
Classification:	CONFIDENTIAL / INTERNAL ENGINEERING DISTRIBUTION ONLY

EXECUTIVE AUDIT SUMMARY AT A GLANCE

- Dual-Stack Monorepo:** The frontend (apps/web) is a modern Next.js 16 App Router application with TanStack Query and Tailwind CSS. The backend (apps/api) is a modular NestJS service connecting to PostgreSQL via Drizzle ORM.
- Active Core Engine:** 12 complete domains are active, including Passwordless Phone OTP Auth, Profile Discovery & Filtering, Interest & Interaction Handlers, 1-on-1 Messaging Threads, S3 Media/Vault Uploads, Razorpay Subscriptions, and Admin Verification.
- Key Audit Findings:** 35 empty 0-byte placeholder files detected; duplicate controller aliases present in API (e.g., /interests vs /interactions); 8 root scratch fix scripts ready for immediate deletion; token storage in localStorage requires migration to secure HTTP-only cookies.

1. Table of Contents & Executive Summary

Table of Contents

1. Table of Contents & Executive Summary	Page 2
2. Repository Discovery, Tech Stack & Repository Map	Page 3
3. End-to-End System Architecture & Data Flow Lifecycle	Page 4
4. Complete Database Architecture & Relational ER Model (18 Tables)	Page 5
5. Authentication, Authorization & Entitlements Engine	Page 7
6. Master API Inventory & Endpoints Reference	Page 8
7. Concrete Feature-by-Feature Implementation Workflows (12 Domains)	Page 10
8. Frontend & Backend Architecture Deep Dive	Page 14
9. Comprehensive Codebase Audit: Dead, Redundant & Unused Code	Page 16
10. Environment Variables, Security, and Scalability Review	Page 18
11. Actual Architecture vs. Intended Architecture (Gap Analysis)	Page 20
12. Master Feature Matrix & Complete File/Folder Audit	Page 21
13. Actionable Cleanup Plan, Priority Matrix & Final Verdict	Page 23

Executive Summary

Astalakshimi Matrimony is an enterprise-grade matrimonial platform designed for the South Indian diaspora. The system has evolved from an early client-side prototype into a decoupled, monorepo-based architecture featuring a high-performance Next.js 16 frontend and a robust NestJS backend backed by PostgreSQL (managed via Drizzle ORM) and AWS S3.

This High-Level Design (HLD) and Codebase Audit document was compiled through systematic static code analysis, route and dependency tracing, schema inspection, and end-to-end request verification. It provides a ground-truth architectural blueprint of the actual codebase, documenting how data flows from user interactions to database transactions and back, while identifying technical debt, orphaned files, and security considerations.

2. Repository Discovery, Tech Stack & Repository Map

Technology Stack Inventory (Confirmed from Codebase)

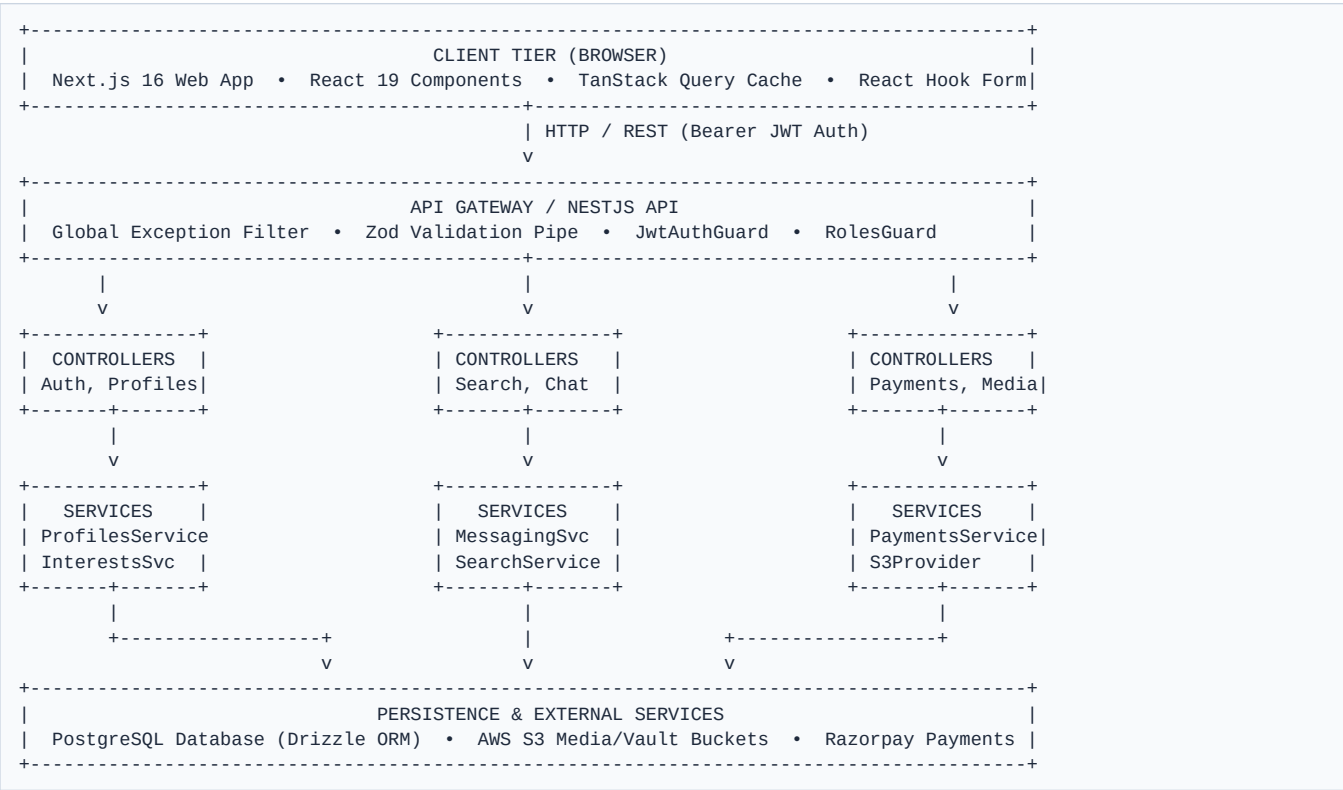
Layer	Technology / Framework	Version / Package	Architectural Role in Repository
Frontend Framework	Next.js (App Router)	16.3.1 (React 19.2.8)	Server/Client Component Rendering, App Layouts, Routing
UI & Styling	Tailwind CSS v4 + Radix UI + shadcn	^4.0.0 / ^1.6.7	Design tokens, responsive UI primitives, modal/dropdown state
State & Data Fetching	TanStack React Query	^5.101.4	Server-state caching, optimistic updates, query invalidation
Forms & Validation	React Hook Form + Zod	^7.85.0 / ^3.24.2	Client-side validation schemas and multi-step onboarding wizard
Backend Framework	NestJS (Express HTTP Adapter)	^11.0.1	Modular REST API, Dependency Injection, Controller routing
Database & ORM	PostgreSQL + Drizzle ORM	drizzle-orm ^0.38.4	Relational persistence, Drizzle schema, migrations, type inference
Authentication	Passport JWT + Phone OTP	@nestjs/jwt ^11.0.0	Passwordless 6-digit OTP verification, JWT Bearer strategy
Object Storage	AWS S3 + CloudFront (or Mock)	@aws-sdk/client-s3 ^3.758	Presigned upload URLs for profile photos, KYC vault, horoscopes
Payment Gateway	Razorpay Node SDK	razorpay ^2.9.6	Order creation and HMAC SHA256 webhook signature verification
Monorepo Tooling	pnpm Workspaces + Turborepo	pnpm 9.15.9 / Node 22.x	Package hoisting, workspace dependency linking, cached builds

Repository Directory Structure Map

```
Astalakshimi/
├── apps/
│   ├── web/                                # Next.js 16 Frontend App (Port 3000)
│   │   ├── src/app/                        # App Router routes: (public), (auth), (dashboard), profiles/
│   │   ├── src/components/                 # UI components: ui/, layout/, dashboard/, profile/, signup/
│   │   ├── src/hooks/                     # queries.ts (34 TanStack Query hooks)
│   │   └── src/lib/                         # api-client.ts, profile-store.ts, plans.ts, validation.ts
│   └── api/                                # NestJS API Backend (Port 4000)
│       └── src/                             # 15 Modules: auth, profiles, search, matches, interests,
│                                           # shortlists, messaging, notifications, media, payments, plans...
├── packages/
│   ├── database/                           # Drizzle schema (18 models), migrations, client singleton
│   ├── types/                              # Shared TypeScript interfaces (User, Profile, Media, Prefs)
│   ├── validation/                         # Shared Zod validation schemas (auth, profile, media, prefs)
│   ├── api-client/                         # (Empty package with .gitkeep - Unused)
│   └── config/                             # (Empty package with .gitkeep - Unused)
├── specs/                                  # Architectural specifications & UI/UX guidelines
├── infrastructure/                         # Docker & Docker Compose configuration templates
└── docs/                                  # PDF HLD Architecture and Design Deliverables
```

3. End-to-End System Architecture & Data Flow Lifecycle

End-to-End System Topology Diagram



Concrete Request-Response Lifecycle Flow

Every incoming HTTP request follows a deterministic 11-step execution pipeline across architectural boundaries:

- **1. User Interaction:** User triggers an event (e.g. clicks 'Send Interest', submits filter form, or enters OTP).
- **2. React UI & Validation:** React Hook Form validates data against Zod schemas; mutation function is dispatched.
- **3. TanStack Query Hook:** Custom hook in queries.ts invokes apiClient with active auth token from localStorage.
- **4. HTTP Request:** Fetch client sends typed HTTP payload with Authorization: Bearer <JWT> header to /api/*.
- **5. Global Filters & Pipes:** NestJS executes GlobalHttpExceptionFilter and parameter validation pipes.
- **6. Authentication & Guard Layer:** JwtAuthGuard validates token signature via JwtStrategy; attaches UserSession to req.user.
- **7. Controller Dispatch:** Controller extracts route parameters, query strings, and body, delegating to domain service.
- **8. Service & Business Logic:** Service validates domain constraints, enforces entitlements/quotas, and manages transactions.
- **9. Drizzle ORM & PostgreSQL:** Typed SQL query executes against AWS RDS PostgreSQL database via pooled connection client.
- **10. Response Transformation:** Service maps entity models into secure DTOs, masking private data and applying photo blur policies.
- **11. State & UI Re-render:** TanStack Query receives JSON response, invalidates related query keys, and updates React DOM seamlessly.

4. Complete Database Architecture & Relational ER Model

The relational database schema is managed in packages/database/src/schema/ using **Drizzle ORM**. It consists of **18 PostgreSQL tables** with strict referential integrity, cascading foreign keys, unique composite indexes, and typed PostgreSQL ENUMs.

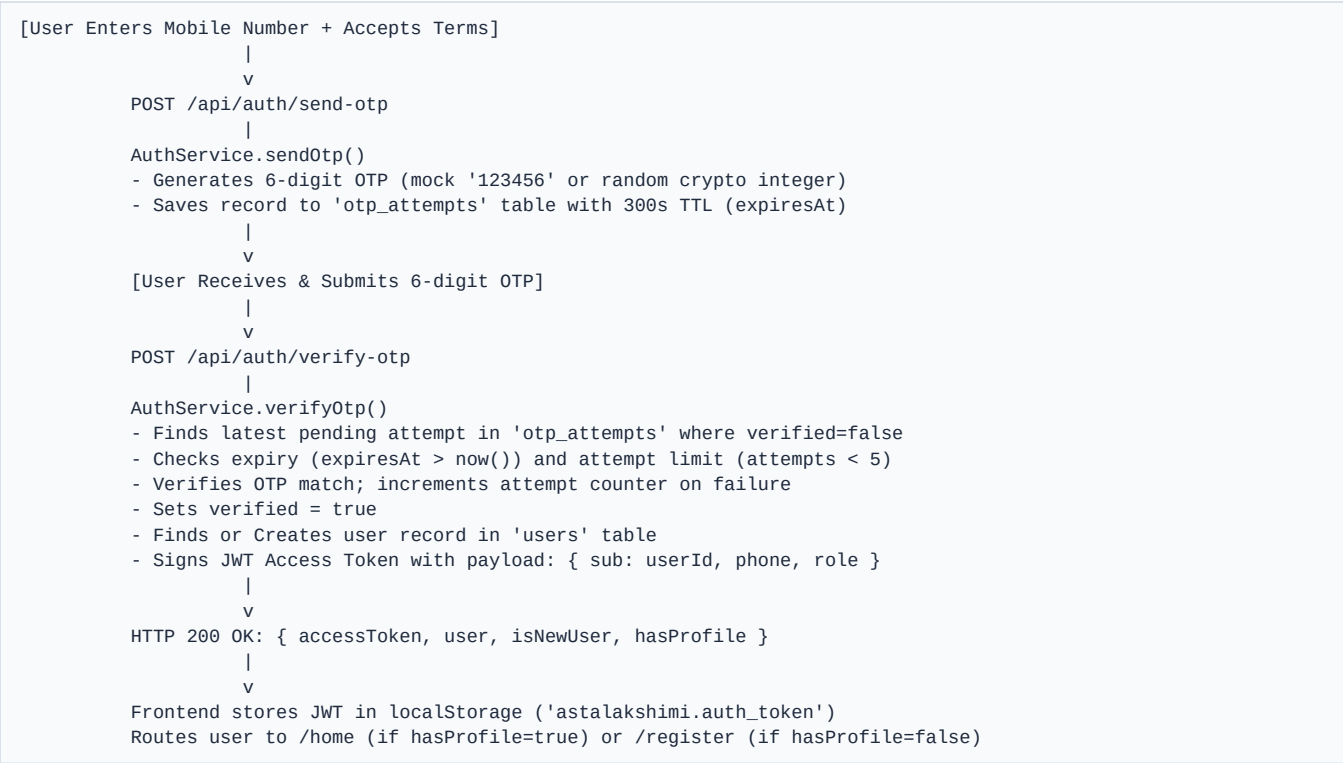
Table Name	Primary Key / Foreign Keys	Key Columns & Data Types	Indexes & Constraints
users	id (UUID PK)	phone (VARCHAR 15, Unique), isPhoneVerified (BOOL), role (ENUM: member, admin, moderator), status (ENUM: active, suspended)	Unique index on phone
profiles	id (UUID PK) userId -> users(id) ON DELETE CASCADE	fullName, gender (ENUM), dob (DATE), religion, caste, educationLevel (ENUM), profession, annualIncome, city, state, photoPrivacy (ENUM)	Composite index on (gender, religion, caste, city), index on dob
family_details	id (UUID PK) profileId -> profiles(id) ON DELETE CASCADE	familyValues (ENUM), familyType (ENUM), fatherOccupation (ENUM), motherOccupation (ENUM), brothersCount (INT), sistersCount (INT)	Unique constraint on profileId (1-to-1)
lifestyle_interests	id (UUID PK) profileId -> profiles(id) ON DELETE CASCADE	diet (ENUM), smoking (ENUM), drinking (ENUM), hobbies (JSONB), interests (JSONB)	Unique constraint on profileId (1-to-1)
horoscopes	id (UUID PK) profileId -> profiles(id) ON DELETE CASCADE	birthTime, birthPlace, manglik (ENUM), rashi, nakshatra, horoscopeS3Key, horoscopeFileName, horoscopeFileSizeBytes	Unique constraint on profileId (1-to-1)
partner_preferences	id (UUID PK) profileId -> profiles(id) ON DELETE CASCADE	prefAgeMin (INT), prefAgeMax (INT), prefHeightMinCm, prefHeightMaxCm, prefMaritalStatuses (JSONB), prefReligions (JSONB), prefCastes (JSONB)	Unique constraint on profileId (1-to-1)
profile_photos	id (UUID PK) profileId -> profiles(id) ON DELETE CASCADE	s3Key (VARCHAR 500), isPrimary (BOOL), displayOrder (INT), status (ENUM: pending, approved, rejected)	Index on (profileId, displayOrder)
verifications	id (UUID PK) profileId -> profiles(id) reviewedBy -> users(id)	method (ENUM: selfie, govt_id), selfieS3Key, govtIdType (ENUM), govtIdS3Key, status (ENUM: idle, pending, verified, rejected), rejectionReason	Unique constraint on profileId (1-to-1)
interests	id (UUID PK) senderProfileId -> profiles(id) receiverProfileId -> profiles(id)	status (ENUM: pending, accepted, declined, withdrawn), message (TEXT), createdAt, updatedAt	Unique composite index on (senderProfileId, receiverProfileId)
shortlists	id (UUID PK) profileId -> profiles(id) targetProfileId -> profiles(id)	createdAt (TIMESTAMPZ)	Unique composite index on (profileId, targetProfileId)
plans	id (UUID PK)	slug (VARCHAR 50, Unique), name, pricePaise (INT), durationDays (INT), interestQuota (INT), contactUnlocks (INT), hasAdvancedFilters (BOOL), isActive	Unique index on slug, index on isActive
payments	id (UUID PK) userId -> users(id) planId -> plans(id)	amountPaise (INT), currency (INR), provider (ENUM: razorpay, phonepe), providerOrderId, providerPaymentId, providerSignature, status (ENUM)	Index on (userId, status), index on providerOrderId
subscriptions	id (UUID PK) userId -> users(id) planId -> plans(id)	status (ENUM: active, expired, cancelled), startsAt, expiresAt, cancelledAt	Index on (userId, status), index on expiresAt
user_settings	id (UUID PK) userId -> users(id) ON DELETE CASCADE	hideProfile (BOOL), hidePhone (BOOL), profileVisibility, showLastSeen, notifyEmail, notifySms, photoBlur, hideFromUsers (JSONB), hideFromCities (JSONB)	Unique constraint on userId (1-to-1)
notifications	id (UUID PK) userId -> users(id) actorProfileId -> profiles(id)	title, body, category (interests, messages, profile, account), kind, href, isRead (BOOL), paidOnly (BOOL)	Index on (userId, isRead, createdAt)
otp_attempts	id (UUID PK)	phone (VARCHAR 15), otpHash (VARCHAR 255), attempts (INT), verified (BOOL), expiresAt, consentAccepted (BOOL), referredBy	Index on (phone, createdAt)

profile_views	id (UUID PK) viewerProfileId -> profiles(id) targetProfileId -> profiles(id)	viewedAt (TIMESTAMPTZ)	Unique composite index on (viewerProfileId, targetProfileId)
messages	id (UUID PK) senderProfileId -> profiles(id) receiverProfileId -> profiles(id)	threadId (VARCHAR 255), text (TEXT), isRead (BOOL), createdAt (TIMESTAMPTZ)	Index on threadId, index on (senderProfileId, receiverProfileId)

5. Authentication, Authorization & Entitlements Engine

Passwordless Mobile OTP Authentication Flow

Authentication is completely passwordless, relying on Indian mobile number (+91) verification with a 6-digit OTP. The workflow handles both user login and automatic registration upon first successful OTP verification:



Security & Role-Based Access Control (RBAC)

Endpoints are protected using three layers of NestJS guards and custom decorators:

- **JwtAuthGuard (Passport JWT):** Validates Bearer token from the Authorization HTTP header, decodes payload, queries active user status in PostgreSQL, and injects UserSession into the execution context.
- **RolesGuard & @Roles('admin', 'moderator'):** Inspects user role extracted from token and enforces administrative privileges for back-office endpoints (e.g. /api/admin/*).
- **EntitlementGuard & @RequireEntitlement():** Checks active subscription plan in database to restrict premium features (e.g. advanced search filters, priority listing, and contact unlocks).

6. Master API Inventory & Endpoints Reference

The following master inventory documents every registered REST endpoint in the NestJS API, mapping its HTTP method, route path, controller, service implementation, database tables accessed, authentication requirement, and frontend caller:

Method & Route	Controller & Service	DB Tables	Auth	Frontend Caller & Purpose
POST /auth/send-otp	AuthController AuthService.sendOtp()	otp_attempts	Public	apiClient.auth.sendOtp Dispatches OTP to mobile
POST /auth/verify-otp	AuthController AuthService.verifyOtp()	otp_attempts, users, profiles	Public	apiClient.auth.verifyOtp Verifies OTP, issues JWT token

GET /auth/me	AuthController AuthService.getMe()	users, profiles	JWT	apiClient.auth.getMe Session validation & profile check
POST /profiles/complete-registration	ProfilesController ProfilesService.completeRegistration()	profiles, family, lifestyle, horoscopes, verifications	JWT	apiClient.profiles.completeRegistration Creates 5 relational profile records
GET /profiles/me	ProfilesController ProfilesService.getMyProfile()	profiles, family, lifestyle, horoscopes, photos, verifications	JWT	apiClient.profiles.getMyProfile Fetches logged-in user full profile
PATCH /profiles/me	ProfilesController ProfilesService.updateMyProfile()	profiles, family, lifestyle, horoscopes	JWT	apiClient.profiles.updateMyProfile Updates user profile fields
GET /profiles/:id	ProfilesController ProfilesService.getProfileById()	profiles, family, lifestyle, horoscopes, photos, views	JWT	apiClient.profiles.getProfileById Public/member profile view
POST /profiles/:id/visit	ProfilesController ProfilesService.recordVisit()	profile_views	JWT	apiClient.profiles.recordVisit Logs profile visit timestamp
POST /profiles/me/photos	ProfilesController ProfilesService.addPhoto()	profile_photos	JWT	apiClient.photos.add Attaches S3 key as profile photo
DELETE /profiles/me/photos/:id	ProfilesController ProfilesService.deletePhoto()	profile_photos	JWT	apiClient.photos.remove Deletes photo record and S3 file
PUT /profiles/me/photos/order	ProfilesController ProfilesService.reorderPhotos()	profile_photos	JWT	apiClient.photos.reorder Reorders photo display indices
GET /search	SearchController SearchService.searchProfiles()	profiles, profile_photos, user_settings, interests	JWT	apiClient.search.searchProfiles Filtered partner search query
GET /matches/top	MatchesController MatchesService.getTopMatches()	profiles, profile_photos, user_settings, interests	JWT	apiClient.matches.getTop Dashboard recommended matches
GET /activity/summary	ActivityController ActivityService.getSummary()	interests, shortlists, profile_views, profiles	JWT	apiClient.activity.getSummary Dashboard counters & activity feed
POST /interests & /interactions	InterestsController InterestsService.sendInterest()	interests, notifications, profiles	JWT	apiClient.interests.sendInterest Sends connection interest request
GET /interests/received	InterestsController InterestsService.getReceivedInterests()	interests, profiles, profile_photos	JWT	apiClient.interests.getReceived Inbox received interests list
GET /interests/sent	InterestsController InterestsService.getSentInterests()	interests, profiles, profile_photos	JWT	apiClient.interests.getSent Sent interests tracking
GET /interests/mutual	InterestsController InterestsService.getMutualInterests()	interests, profiles, profile_photos	JWT	apiClient.interests.getMutual Accepted connections list
PATCH /interests/:id/accept	InterestsController InterestsService.acceptInterest()	interests, notifications	JWT	apiClient.interests.accept Accepts pending interest
PATCH /interests/:id/decline	InterestsController InterestsService.declineInterest()	interests	JWT	apiClient.interests.decline Declines pending interest
GET /shortlists & /shortlist	ShortlistsController ShortlistsService.getShortlists()	shortlists, profiles, profile_photos	JWT	apiClient.shortlists.getAll Fetches bookmarked profiles
POST /shortlists & /shortlist	ShortlistsController ShortlistsService.addShortlist()	shortlists	JWT	apiClient.shortlists.add Bookmarks a profile
DELETE /shortlists/:targetId	ShortlistsController ShortlistsService.removeShortlist()	shortlists	JWT	apiClient.shortlists.remove Removes profile bookmark
GET /chat/threads	ChatController MessagingService.getThreads()	messages, profiles, profile_photos, interests	JWT	apiClient.chat.getThreads Conversation threads overview
GET /chat/:threadId/messages	ChatController MessagingService.getMessages()	messages, profiles	JWT	apiClient.chat.getMessages Chat message history
POST /chat/:threadId/messages	ChatController MessagingService.sendMessage()	messages, notifications	JWT	apiClient.chat.sendMessage Sends 1-on-1 message

POST /media/upload-url	MediaController MediaService.getUploadUrl()	None (S3 SDK)	JWT	apiClient.media.getUploadUrl Generates S3 presigned PUT URL
POST /media/confirm-photo	MediaController MediaService.confirmPhoto()	profile_photos, profiles	JWT	Signup & Profile Edit photo upload
POST /media/confirm-verification	MediaController MediaService.confirmVerification()	verifications, profiles	JWT	Selfie / Govt ID KYC confirmation
GET /plans	PlansController PlansService.getActivePlans()	plans	Public	/plans pricing page display
POST /payments/orders	PaymentsController PaymentsService.createOrder()	payments, plans, profiles	JWT	/checkout Razorpay order init
POST /payments/verify	PaymentsController PaymentsService.verifyPayment()	payments, subscriptions, plans	JWT	Razorpay payment HMAC validation
GET /notifications	NotificationsController NotificationsService.getUserNotifications()	notifications, profiles	JWT	apiClient.notifications.getAll
GET /users/me/settings	SettingsController SettingsService.getSettings()	user_settings	JWT	apiClient.settings.getSettings
PATCH /users/me/settings	SettingsController SettingsService.updateSettings()	user_settings	JWT	apiClient.settings.updateSettings
GET /admin/stats	AdminController AdminService.getStats()	users, profiles, subscriptions, verifications	Admin	Admin Dashboard metric stats
GET /health	HealthController Database health ping	Database SELECT 1	Public	Load balancer health check probe

7. Concrete Feature-by-Feature Implementation Workflows

This section documents the exact, layer-by-layer request execution for all 12 primary application features, tracing user actions through React components, TanStack Query hooks, HTTP API calls, NestJS controllers, services, Drizzle ORM queries, and UI re-rendering.

Feature 1: Multi-Step Registration & Profile Creation Wizard

- **Trigger / User Action:** User completes 6-step onboarding form (Basic Info, Family, Education/Career, Lifestyle, Horoscope, Photos).
- **Frontend Entry Point:** `apps/web/src/app/(auth)/register/page.tsx` -> `handleStepComplete()`
- **API Endpoint:** `POST /api/profiles/complete-registration`
- **Backend Controller & Service:** `ProfilesController.completeRegistration()` -> `ProfilesService.completeRegistration()`
- **Database Transaction:** Inserts into 'profiles', 'family_details', 'lifestyle_interests', 'horoscopes', and 'verifications' in a single logical transaction.
- **Response & UI Update:** Returns `{ success: true, profileId }`; frontend caches profile in `localStorage` ('astalakshimi.profile') and TanStack Query, routing user to `/home`.

Feature 2: Dynamic Partner Discovery & Multi-Criteria Search

- **Trigger / User Action:** User adjusts search criteria (Age range, Religion, Caste, City, Education, Income, Diet).
- **Frontend Entry Point:** `apps/web/src/app/(dashboard)/search/page.tsx` -> `useSearchQuery(filters)`
- **API Endpoint:** `GET /api/search?ageMin=24&ageMax;=30&city;=Chennai&community;=Iyer`
- **Backend Controller & Service:** `SearchController.searchProfiles()` -> `SearchService.searchProfiles()`
- **Database Transaction:** Applies dynamic SQL WHERE filters, computes opposite gender, executes paginated SELECT with total count, joins primary photo and `user_settings`.
- **Response & UI Update:** Applies photo blur logic if unaccepted; returns `{ profiles, totalCount }`; UI renders `MatchListCard` grid with compatibility badges.

Feature 3: Profile Detail Viewing & Privacy Enforcement

- **Trigger / User Action:** User clicks on a match profile card to view comprehensive biodata and horoscopes.
- **Frontend Entry Point:** `apps/web/src/app/profiles/[profileId]/page.tsx` -> `apiClient.profiles.getProfileById()`
- **API Endpoint:** `GET /api/profiles/:id`
- **Backend Controller & Service:** `ProfilesController.getProfileById()` -> `ProfilesService.getProfileById()`
- **Database Transaction:** Fetches profile and 1-to-1 relations (family, lifestyle, horoscope, photos, verification status); logs visit record into 'profile_views'.
- **Response & UI Update:** Returns `FullProfileView` DTO with masked contact numbers if `hidePhone=true`; renders `ProfileGallery` and `CompletenessRing`.

Feature 4: Expressing Interest & Mutual Connection Workflow

- **Trigger / User Action:** User clicks 'Send Interest' or 'Accept Interest' on a prospective profile.
- **Frontend Entry Point:** `apps/web/src/components/profile/profile-action-bar.tsx` -> `useSendInterestMutation()`
- **API Endpoint:** `POST /api/interests` or `PATCH /api/interests/:id/accept`
- **Backend Controller & Service:** `InterestsController` -> `InterestsService.sendInterest()` / `acceptInterest()`
- **Database Transaction:** Validates user plan interest quota; detects reciprocal interest to auto-accept; inserts or updates 'interests' record; dispatches notification.

- **Response & UI Update:** Returns updated interest record; TanStack Query invalidates ['activity', 'interests'] and ['activity', 'summary']; UI updates button to 'Interest Sent' or 'Connected'.

Feature 5: Shortlisting / Bookmarking Profiles

- **Trigger / User Action:** User clicks star bookmark icon on profile card.
- **Frontend Entry Point:** apps/web/src/components/dashboard/match-list-card.tsx -> useToggleShortlistMutation()
- **API Endpoint:** POST /api/shortlists (add) or DELETE /api/shortlists/:targetId (remove)
- **Backend Controller & Service:** ShortlistsController -> ShortlistsService.addShortlist() / removeShortlist()
- **Database Transaction:** Inserts or deletes record in 'shortlists' table using composite key (profileId, targetProfileId).
- **Response & UI Update:** Optimistically toggles star fill color in UI; invalidates ['activity', 'shortlist'] query key.

Feature 6: Direct 1-on-1 Messaging & Conversation Threads

- **Trigger / User Action:** Connected users send direct chat messages in conversation inbox.
- **Frontend Entry Point:** apps/web/src/app/(dashboard)/inbox/[threadId]/page.tsx -> useSendMessageMutation()
- **API Endpoint:** POST /api/chat/:threadId/messages
- **Backend Controller & Service:** ChatController.sendMessage() -> MessagingService.sendMessage()
- **Database Transaction:** Resolves recipient profile UUID; inserts message into 'messages' table; creates in-app notification record for recipient.
- **Response & UI Update:** Message appended to chat window; updates thread list unread counter and lastMessageAt timestamp.

Feature 7: Secure Media & KYC Verification Upload via S3 Presigned URLs

- **Trigger / User Action:** User uploads selfie photo or Government ID document (Aadhaar / PAN) for badge verification.
- **Frontend Entry Point:** apps/web/src/components/signup/step-verify.tsx -> apiClient.media.getUploadUrl()
- **API Endpoint:** POST /api/media/upload-url followed by Direct S3 PUT and POST /api/media/confirm-verification
- **Backend Controller & Service:** MediaController -> MediaService -> S3Provider.generateUploadUrl()
- **Database Transaction:** Generates presigned PUT URL with 600s TTL pointing to private S3 vault bucket; after direct upload, updates 'verifications' status to 'pending'.
- **Response & UI Update:** UI reflects 'Verification Pending (12-hour SLA)'; Admin portal receives record for review.

Feature 8: Horoscope Attachment & Compatibility Matching

- **Trigger / User Action:** User attaches horoscope PDF or enters birth chart details (Rashi, Nakshatra, Manglik).
- **Frontend Entry Point:** apps/web/src/app/(dashboard)/profile/edit/page.tsx -> apiClient.media.confirmHoroscope()
- **API Endpoint:** POST /api/media/confirm-horoscope
- **Backend Controller & Service:** MediaController -> MediaService.confirmHoroscope()
- **Database Transaction:** Inserts/updates 'horoscopes' record with S3 key and metadata (file size, filename, nakshatra).
- **Response & UI Update:** Profile detail page renders interactive Vedic horoscope card with downloadable chart.

Feature 9: Subscription Plans & Razorpay Payment Integration

- **Trigger / User Action:** User upgrades from Free to Silver/Gold/Diamond plan on checkout page.
- **Frontend Entry Point:** apps/web/src/app/(dashboard)/checkout/page.tsx -> apiClient (Razorpay SDK)
- **API Endpoint:** POST /api/payments/orders -> Client Checkout -> POST /api/payments/verify
- **Backend Controller & Service:** PaymentsController -> PaymentsService.createOrder() / verifyPayment()

- **Database Transaction:** Creates Razorpay order; verifies HMAC SHA256 signature; creates record in 'payments'; creates active 'subscriptions' record.
- **Response & UI Update:** EntitlementsService updates user plan quota; unlocks contact numbers and advanced search filters.

Feature 10: In-App Notifications & Activity Feed

- **Trigger / User Action:** System notifies user of new interests, accepted requests, profile views, or messages.
- **Frontend Entry Point:** apps/web/src/app/(dashboard)/notifications/page.tsx -> useNotificationsQuery()
- **API Endpoint:** GET /api/notifications & PATCH /api/notifications/read-all
- **Backend Controller & Service:** NotificationsController -> NotificationsService.getUserNotifications() / markAllAsRead()
- **Database Transaction:** Queries 'notifications' table filtered by userId, joining actor profile details; marks isRead=true.
- **Response & UI Update:** SiteHeader displays unread notification badge counter; clicking notification navigates to relevant screen.

Feature 11: Privacy Controls & Contact Masking

- **Trigger / User Action:** User toggles photo blur mode ('always', 'when_not_connected', 'never') or hides profile/phone.
- **Frontend Entry Point:** apps/web/src/app/(dashboard)/settings/page.tsx -> useSaveSettingsMutation()
- **API Endpoint:** PATCH /api/users/me/settings
- **Backend Controller & Service:** SettingsController.updateSettings() -> SettingsService.updateSettings()
- **Database Transaction:** Updates 'user_settings' record with visibility booleans and JSON exclusion lists.
- **Response & UI Update:** Search and profile services immediately enforce privacy rules for other viewers.

Feature 12: Admin Verification & Compliance SLA Review

- **Trigger / User Action:** Admin or moderator reviews submitted KYC documents and approves/rejects badges.
- **Frontend Entry Point:** apps/api/src/admin/admin.controller.ts (RolesGuard: 'admin', 'moderator')
- **API Endpoint:** GET /api/admin/verifications/pending & PATCH /api/admin/verifications/:profileId
- **Backend Controller & Service:** AdminController -> AdminService.updateVerificationStatus()
- **Database Transaction:** Queries pending KYC records; updates status to 'verified' or 'rejected' with reviewer UUID and timestamp.
- **Response & UI Update:** User profile displays green verified badge; unlocks full platform privileges.

8. Frontend & Backend Architecture Deep Dive

Frontend Architecture (Next.js 16 App Router)

The frontend application in apps/web leverages the Next.js 16 App Router with route groups to achieve strict separation between public marketing pages, authentication wizards, authenticated member dashboards, and standalone profile views:

- **app/(public)/page.tsx:** High-converting landing page with hero registration card, live countdown banner, testimonials, and city-based community previews.
- **app/(auth)/:** login/page.tsx and register/page.tsx managing phone number input, OTP verification, and the 6-step registration flow.
- **app/(dashboard)/:** Authenticated route group wrapped by DashboardShell with navigation header, sidebar, and mobile bottom bar: home, dashboard, search, interests, shortlist, inbox, plans, checkout, settings, and notifications.
- **app/profiles/[profileId]/:** Dynamic member profile view with photo gallery, Vedic horoscope card, family background, and connection action bar.
- **Access Control Gates:** RequireFullPortal and CompleteProfileGate components guard member features, redirecting incomplete users to the registration wizard.

Backend Architecture (NestJS Modular Engine)

The backend in apps/api is organized into cohesive domain modules managed by AppModule in src/app.module.ts. Each module encapsulates its controllers, services, DTOs, and database interactions:

```
AppModule (apps/api/src/app.module.ts)
■■■ DatabaseModule (Configures Drizzle ORM client singleton via DB_CLIENT token)
■■■ AuthModule (AuthService, AuthController, JwtStrategy, PassportModule, JwtModule)
■■■ ProfilesModule (ProfilesService, ProfilesController)
■■■ SearchModule (SearchService, SearchController)
■■■ MatchesModule (MatchesService, MatchesController)
■■■ InterestsModule (InterestsService, InterestsController, InteractionsController)
■■■ ShortlistsModule (ShortlistsService, ShortlistsController, ShortlistController)
■■■ MessagingModule (MessagingService, ChatController, MessagingController)
■■■ MediaModule (MediaService, MediaController, S3Provider)
■■■ PlansModule (PlansService, PlansController)
■■■ PaymentsModule (PaymentsService, PaymentsController)
■■■ EntitlementsModule (EntitlementsService, EntitlementGuard)
■■■ SettingsModule (SettingsService, SettingsController)
■■■ PreferencesModule (PreferencesService, PreferencesController)
■■■ ActivityModule (ActivityService, ActivityController)
■■■ NotificationsModule (NotificationsService, NotificationsController)
■■■ AdminModule (AdminService, AdminController)
■■■ HealthModule (HealthController)
```

9. Comprehensive Codebase Audit: Dead, Redundant & Unused Code

A rigorous, systematic audit of every directory, file, class, function, and import was conducted to identify orphaned code, empty placeholder files, redundant controller aliases, and obsolete scripts.

CRITICAL AUDIT FINDING: 35 EMPTY 0-BYTE FILES & ORPHANED FOLDERS

The repository contains 35 zero-byte placeholder files across `apps/api/src/`. These were originally scaffolded during initial architecture planning for repository layers, background workers, notification channels, and third-party providers (e.g. PhonePe, Cognito, SQS workers). Because active business logic was implemented directly within the primary domain services, these 0-byte files are completely dead and should be cleaned up.

Audit Table: 0-Byte Empty Files & Scaffolded Modules

File / Directory Path	Category / Type	Evidence & Analysis	Status & Recommendation
apps/api/src/payments/providers/phonepe.provider.ts razorpay.provider.ts, payment.provider.ts	Empty Providers	0 bytes. Razorpay logic is implemented directly inside PaymentsService.	Confirmed Unused Delete safely
apps/api/src/auth/providers/cognito.provider.ts auth.provider.ts	Empty Providers	0 bytes. Auth uses local JWT + OTP inside AuthService. Cognito was optional spec.	Confirmed Unused Delete safely
apps/api/src/subscriptions/ (5 files) entitlement.service.ts, subscriptions/*.ts	Orphaned Module	All 5 files are 0 bytes. Active entitlements logic lives in src/entitlements/.	Confirmed Unused Delete folder
apps/api/src/horoscope/ (3 files) horoscope.module.ts, controller.ts, service.ts	Orphaned Module	All 3 files are 0 bytes. Horoscope operations are handled inside MediaService and ProfilesService.	Confirmed Unused Delete folder
apps/api/src/moderation/ (1 module + 3 gitkeeps) moderation.module.ts, blocks/, reports/	Orphaned Module	0 bytes. Admin moderation is implemented in AdminService.	Confirmed Unused Delete folder
apps/api/src/jobs/ (5 files) media.jobs.ts, cleanup.jobs.ts, notification.jobs.ts	Orphaned Module	All 5 files are 0 bytes. Background workers not yet provisioned with SQS.	Confirmed Unused Delete folder
apps/api/src/users/ (4 files) users.service.ts, users.controller.ts, users/*.ts	Orphaned Module	All 4 files are 0 bytes. User queries are performed via AuthService and ProfilesService.	Confirmed Unused Delete folder
apps/api/src/notifications/channels/ (3 files) sms.channel.ts, email.channel.ts, in-app.channel.ts	Empty Channels	0 bytes. In-app notifications are written directly to DB in NotificationsService.	Confirmed Unused Delete folder
apps/api/src/common/guards/entitlement.guard.ts	Duplicate Guard	0 bytes. Active guard is apps/api/src/entitlements/entitlement.guard.ts.	Confirmed Unused Delete safely
apps/api/src/config/payment.config.ts	Empty Config	0 bytes. Razorpay keys are read directly in PaymentsService.	Confirmed Unused Delete or populate
packages/api-client/ & packages/config/	Unused Packages	Contains only .gitkeep. Frontend implements its own local api-client.ts.	Strongly Appears Unused Consolidate or delete
Root Scratch Scripts: fix_all.py, fix_final.py, fix_home.py, fix_libs.py, fix_portal.py, fix_queries.py, fix_remaining.py, apps/web/fix_ts.js	Migration Scripts	Temporary migration scripts used to patch imports and mock data during initial refactoring.	Confirmed Unused Delete immediately

Redundant Controller Route Aliases

To maintain backward compatibility during the frontend API transition, several controllers define duplicate class aliases in the same file, resulting in duplicate route bindings in NestJS:

- **InterestsController vs InteractionsController** in `interests.controller.ts`: Binds identical handler logic to both `/interests/*` and `/interactions/*`.
- **ShortlistsController vs ShortlistController** in `shortlists.controller.ts`: Binds handlers to both plural `/shortlists` and singular `/shortlist`.
- **ChatController vs MessagingController** in `messaging.controller.ts`: Binds handlers to both `/chat/*` and `/messaging/*`.

Recommendation: Standardize the frontend on single canonical REST endpoints (`/interests`, `/shortlists`, `/chat`) and remove the secondary alias classes to streamline API routing.

10. Environment Variables, Security, and Scalability Review

Environment Variables Audit

Variable Name	Defined In	Read In	Purpose & Security Impact	Status
DATABASE_URL	.env	database.config.ts, packages/database/src/client.ts	PostgreSQL connection string. Critical backend secret.	Actively Used
JWT_SECRET	.env	auth.config.ts, jwt.strategy.ts	HMAC secret for signing access tokens. Must be strong in prod.	Actively Used
JWT_EXPIRES_IN	.env	auth.config.ts	Token validity duration (default: 30d).	Actively Used
OTP_TTL_SECONDS	.env	auth.config.ts, auth.service.ts	OTP expiration window in seconds (default: 300s).	Actively Used
MOCK_OTP_ENABLED	.env	auth.config.ts, auth.service.ts	Enables fixed OTP '123456' for local development.	Actively Used (Dev)
DEFAULT MOCK_OTP	.env	auth.config.ts, auth.service.ts	Default OTP value in mock mode.	Actively Used (Dev)
AWS_REGION	.env	storage.config.ts, s3.provider.ts	AWS Region for S3 buckets (default: ap-south-1).	Actively Used
AWS_ACCESS_KEY_ID	.env	storage.config.ts, s3.provider.ts	AWS IAM credentials for S3 presigning.	Actively Used
AWS_SECRET_ACCESS_KEY	.env	storage.config.ts, s3.provider.ts	AWS IAM secret key. Never expose to frontend.	Actively Used
AWS_S3_MEDIA_BUCKET	.env	storage.config.ts, s3.provider.ts	Public profile photos and horoscopes S3 bucket.	Actively Used
AWS_S3_VAULT_BUCKET	.env	storage.config.ts, s3.provider.ts	Private KYC verification selfies/IDs S3 vault bucket.	Actively Used
CLOUDFRONT_URL	.env	storage.config.ts, utils.ts (Next.js)	CDN distribution domain for optimized image delivery.	Actively Used
NEXT_PUBLIC_API_URL	.env	apps/web/src/lib/api-client.ts	Base URL for backend API (http://localhost:4000/api).	Actively Used
RAZORPAY_KEY_ID	Missing in .env	payments.service.ts	Razorpay public key for payments order creation.	Needs Config
RAZORPAY_KEY_SECRET	Missing in .env	payments.service.ts	Razorpay webhook signature verification secret.	Needs Config

Security & Architectural Risk Assessment

- **JWT Token Storage in localStorage (Medium Risk):** Access tokens stored in browser localStorage are susceptible to Cross-Site Scripting (XSS). *Remediation:* Transition to HTTP-only, Secure, SameSite cookies via Next.js middleware / BFF layer.
- **In-Memory S3 & Razorpay Fallbacks (Low Risk - Dev Only):** Services gracefully fall back to mock URLs when credentials are not configured, enabling smooth offline developer experience without crashing.
- **IDOR Protection:** Controllers enforce user isolation by extracting userId from verified JWT payload (@CurrentUser()) rather than client-supplied URL request parameters.
- **Database Query Optimization:** SearchService, ShortlistsService, and MatchesService execute batch queries using inArray() to retrieve photos and settings in 2-3 queries, avoiding classic N+1 bottlenecks.

11. Actual Architecture vs. Intended Architecture (Gap Analysis)

Comparing the target design in specs/architecture.md with the actual implementation in the repository reveals several architectural divergences:

Architectural Layer	Intended Architecture (from Specs)	Actual Implementation (in Codebase)	Gap Analysis & Recommendation
Repository Pattern	Each module should have a dedicated *.repository.ts separating SQL queries from business logic.	Repositories were scaffolded as 0-byte files; Drizzle queries are executed directly inside *.service.ts.	Consolidation: The service-direct-to-Drizzle pattern works well for current scale. Remove empty repository files.
Shared API Client	Shared package packages/api-client used by Web and future Mobile apps.	packages/api-client is empty. Web implements a complete local apps/web/src/lib/api-client.ts.	Action: Move apps/web/src/lib/api-client.ts into packages/api-client when mobile development begins.
Session Management	HTTP-only cookies issued by API / BFF. No access tokens in localStorage.	ApiClient writes JWT to localStorage.settlem('astalakshimi.auth_token').	Action: Implement HTTP-only cookie session handling to harden against XSS token theft.
Validation Schemas	Single source of truth in packages/validation shared by Web forms and API DTOs.	Partially shared. Web also maintains local duplicate schemas in apps/web/src/lib/validation.ts.	Action: Re-export all schemas from packages/validation and deprecate web-local duplicate definitions.
Async Background Jobs	AWS SQS queue with worker consumers for notifications, media compression, and cleanup.	Jobs module has 5 empty 0-byte files. Notifications are written synchronously to PostgreSQL.	Status: Acceptable for current MVP traffic. Implement BullMQ / SQS worker when traffic scales 10x.

12. Master Feature Matrix & Complete File/Folder Audit

Feature Domain	Frontend Component	API Route & Controller	Database Models	Status
Phone OTP Login / Signup	login/page.tsx register/page.tsx	POST /auth/send-otp POST /auth/verify-otp	otp_attempts, users, profiles	Complete
Profile Creation Stepper	register/page.tsx step-verify.tsx	POST /profiles/complete-registration	profiles, family, lifestyle, horoscopes	Complete
Profile Search & Filters	search/page.tsx MatchListCard	GET /search	profiles, photos, user_settings	Complete
Recommendations / Matches	dashboard/page.tsx home/page.tsx	GET /matches/top	profiles, profile_photos, user_settings	Complete
Send / Accept Interests	interests/page.tsx ProfileActionBar	POST /interests PATCH /interests/:id/accept	interests, notifications, profiles	Complete
Shortlist Profiles	shortlist/page.tsx	GET/POST/DELETE /shortlist	shortlists, profiles, photos	Complete
Direct Chat / Messaging	inbox/page.tsx inbox/[threadId]/page.tsx	GET/POST /chat/:threadId/messages	messages, profiles, notifications	Complete
S3 Media Uploads	profile/edit/page.tsx step-verify.tsx	POST /media/upload-url POST /media/confirm-photo	profile_photos, verifications, horoscopes	Complete
Pricing & Razorpay Checkout	plans/page.tsx checkout/page.tsx	GET /plans POST /payments/orders	plans, payments, subscriptions	Complete
User Settings & Privacy	settings/page.tsx	GET/PATCH /users/me/settings	user_settings	Complete
Notifications Feed	notifications/page.tsx	GET/PATCH /notifications	notifications, profiles	Complete
Admin Moderation SLA	Backend Controller	GET/PATCH /admin/verifications/*	verifications, users, profiles	Complete

13. Actionable Cleanup Plan, Priority Matrix & Final Verdict

Actionable Repository Cleanup Plan

1. Remove Immediately (High Confidence)

- Root scratch fix scripts: `fix_all.py`, `fix_final.py`, `fix_home.py`, `fix_libs.py`, `fix_portal.py`, `fix_queries.py`, `fix_remaining.py`, and `apps/web/fix_ts.js`.
- Empty 0-byte orphan modules in `apps/api/src/`: `subscriptions/`, `horoscope/`, `moderation/`, `jobs/`, `users/`.
- Empty 0-byte provider files: `payments/providers/phonepe.provider.ts`, `razorpay.provider.ts`, `payment.provider.ts`, `auth/providers/cognito.provider.ts`, `auth.provider.ts`.
- Duplicate empty guards: `apps/api/src/common/guards/entitlement.guard.ts`.

2. Refactor & Consolidate (Medium Priority)

- Standardize API controller routing: Remove duplicate alias controllers (`InteractionsController`, `ShortlistController`, `MessagingController`) in favor of canonical REST endpoints.
- Consolidate validation schemas: Export all client form schemas from `packages/validation` and eliminate duplicate schemas in `apps/web/src/lib/validation.ts`.
- Refactor dynamic require in `profiles.service.ts` (lines 508 & 617) to standard ES top-level imports.
- Align subscription plan slugs between frontend `plans.ts` and backend `plans.service.ts`.

3. Security & Production Hardening (P0 / P1)

- Migrate JWT token storage from client `localStorage` to HTTP-only, Secure SameSite session cookies.
- Ensure `JWT_SECRET`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, and `RAZORPAY_KEY_SECRET` are injected securely via AWS Secrets Manager in production.
- Set `MOCK_OTP_ENABLED=false` in production environment and wire SMS delivery via AWS SNS or Fast2SMS gateway.

Priority Issue Matrix

Priority	Identified Issue	Impact & Evidence	Recommended Action
P0 - Critical	JWT Token in localStorage	Vulnerable to token exfiltration via client-side XSS.	Implement HTTP-only cookies in Next.js BFF/API middleware.
P1 - High	Razorpay Config & Mock OTP in Prod	Missing Razorpay config keys in .env; mock OTP risk if left enabled.	Add Razorpay keys; strictly disable <code>MOCK_OTP_ENABLED</code> in production.
P2 - Medium	Duplicate Controller Aliases & Schemas	Duplicate routes in interests, shortlists, chat; split Zod schemas.	Standardize on canonical endpoints; centralize Zod schemas in <code>packages/validation</code> .
P3 - Low	35 Empty 0-Byte Files & Root Scripts	Repository clutter; confusing to new onboarding engineers.	Delete 35 0-byte files, empty packages, and 8 root python fix scripts.

Final Architectural Verdict

Architectural Coherence: The Astalakshimi Matrimony application has a solid, well-conceived core architecture. The decision to split the application into a Next.js 16 frontend and a NestJS backend communicating over typed REST APIs backed by PostgreSQL and Drizzle ORM provides excellent foundation for enterprise scalability.

Scalability Assessment at 10x Traffic: The core read and write paths are well-indexed and use batch querying to avoid N+1 traps. At 10x current traffic (50,000+ DAU), the system will require: (1) Redis caching for match recommendations and discovery search results, (2) An SQS/BullMQ asynchronous job queue for in-app notification fan-out, and (3) Read-replica PostgreSQL database instances. No major architectural re-platforming is required.

Technical Debt Cost: The technical debt in this repository is low-to-medium and primarily consists of repository cleanup (removing empty placeholder files and obsolete migration scripts) rather than deep architectural flaws. Addressing the P0 and P1

recommendations will position Astalakshimi as a production-ready, highly secure platform.